

DOCUMENTACIÓN FUNCIONAL Y TÉCNICA — FAMÍLIA+

Versión: 1.0

Estado: Especificación inicial para desarrollo y producción

Idioma de la aplicación: Català

Tipo de aplicación: Web responsive / PWA preparada para móvil, tablet y escritorio

Usuarios: Adultos responsables e hijos/as de una unidad familiar

1. VISIÓN DEL PRODUCTO

Família+ será una herramienta web para gestionar de forma centralizada:

- paga fija mensual;
- dinero disponible;
- ahorro;
- tareas y responsabilidades;
- trabajos extra remunerados;
- recompensas económicas;
- recompensas en tiempo de pantalla;
- bonificaciones;
- penalizaciones;
- tiempo de pantalla;
- objetivos de ahorro;
- aprobaciones;
- movimientos;
- resumen y cierre mensual.

El sistema debe permitir que los hijos visualicen de forma sencilla:

- cuánto dinero tienen;
- cuánto han ahorrado;
- cuánto pueden gastar;
- cuánto tiempo de pantalla tienen;
- qué tareas pueden hacer;
- qué tareas están pendientes de aprobación;
- qué recompensas han conseguido.

El adulto dispondrá de un panel de administración completo.

2. PRINCIPIOS FUNCIONALES

2.1. El adulto mantiene el control

Ninguna acción realizada por un hijo que genere una recompensa económica o de pantalla será efectiva hasta que un adulto la valide.

Flujo:

DISPONIBLE → MARCADA COMO HECHA → PENDIENTE → APROBADA / RECHAZADA

Solo **APROBADA** genera movimientos.

2.2. No se modifican saldos directamente

El sistema no guardará únicamente:

Martí = 34,50 €

Guardará movimientos:

- +14 € paga disponible;
- +6 € paga ahorro;
- +3 € tarea;
- -5 € gasto;
- +2 € bonificación.

Saldo:

SUMA(movimientos)

Esto proporciona trazabilidad y evita descuadres.

2.3. Dinero y pantalla funcionan mediante ledger

Existirán dos sistemas independientes de movimientos:

Money Ledger

Gestiona:

- dinero disponible;

- ahorro.

Screen Time Ledger

Gestiona:

- minutos disponibles;
 - bonificaciones;
 - penalizaciones;
 - consumo.
-

3. ROLES

PARENT

Adulto administrador.

Puede:

- crear hijos;
 - modificar hijos;
 - gestionar configuración;
 - crear tareas;
 - editar tareas;
 - eliminar/desactivar tareas;
 - aprobar tareas;
 - rechazar tareas;
 - gestionar paga;
 - gestionar ahorro;
 - introducir gastos;
 - gestionar pantalla;
 - introducir bonificaciones;
 - introducir penalizaciones;
 - consultar todos los históricos;
 - realizar cierres mensuales.
-

CHILD

Usuario infantil.

Puede:

- consultar su información;

- consultar su dinero;
- consultar ahorro;
- consultar pantalla;
- consultar tareas;
- marcar una tarea como completada;
- consultar estados;
- consultar objetivos de ahorro;
- consultar movimientos.

No puede:

- aprobar tareas;
- editar recompensas;
- alterar saldos;
- modificar reglas;
- modificar tareas;
- consultar otros hijos.

4. ARQUITECTURA

Frontend

- Vue 3
- Vite
- TypeScript
- Vue Router
- Pinia
- Axios
- CSS propio / Tailwind opcional
- Chart.js o Apache ECharts
- PWA opcional

Vue mantiene Vue 3 como arquitectura actual y Vite proporciona el proceso de build optimizado para producción.

Backend

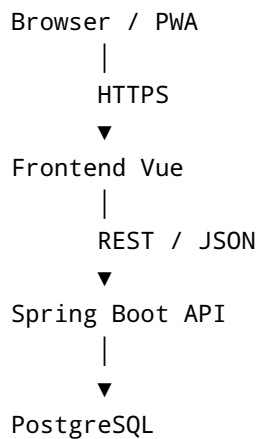
- Java 21
- Spring Boot 4.1.x
- Spring Web
- Spring Data JPA
- Spring Security
- Bean Validation
- PostgreSQL Driver

- Flyway
- Spring Boot Actuator
- Lombok opcional
- JWT
- OpenAPI / Swagger

Base de datos

PostgreSQL 18

Infraestructura



5. ESTRUCTURA DEL REPOSITORIO

Se recomienda **monorepo**.

```
familia-plus/  
|  
├── backend/  
|   ├── src/  
|   ├── pom.xml  
|   ├── Dockerfile  
|   └── .env.example  
|  
├── frontend/  
|   └── src/
```

```
|
|   ├── package.json
|   ├── Dockerfile
|   └── .env.example
|
|   ├── docs/
|   |   ├── architecture/
|   |   ├── database/
|   |   └── api/
|   |
|   ├── docker-compose.yml
|   ├── docker-compose.prod.yml
|   ├── .gitignore
|   ├── README.md
|   └── DEPLOY.md
```

6. ENTIDADES PRINCIPALES

```
Family
|
|   ├── User
|   |   ├── Parent
|   |   └── Child
|   |
|   ├── AllowanceRule
|   ├── MonthlyAllowance
|   |
|   ├── Task
|   |   ├── TaskAssignment
|   |   └── TaskCompletion
|   |
|   ├── MoneyTransaction
|   ├── ScreenTimeTransaction
|   |
|   ├── SavingsGoal
|   |
|   ├── ScreenTimeRule
|   ├── DailyScreenBalance
|   |
|   ├── RewardAdjustment
|   |
|   └── MonthlySettlement
```

7. MODELO DE BASE DE DATOS

7.1. FAMILY

Representa una unidad familiar.

Tabla `families`

Campo	Tipo	Restricción
id	UUID	PK
name	VARCHAR(100)	NOT NULL
currency	VARCHAR(3)	DEFAULT EUR
timezone	VARCHAR(50)	DEFAULT Europe/Madrid
language	VARCHAR(10)	DEFAULT ca
active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL

Ejemplo:

```
Família Sande-Lima
EUR
Europe/Madrid
ca
```

7.2. USERS

Todos los usuarios autenticables.

Tabla `users`

Campo	Tipo
id	UUID PK
family_id	UUID FK
email	VARCHAR(255)
username	VARCHAR(100)
password_hash	VARCHAR(255)
role	ENUM
active	BOOLEAN
last_login_at	TIMESTAMP
created_at	TIMESTAMP
updated_at	TIMESTAMP

Role

PARENT
CHILD

Para menores puede utilizarse un nombre de usuario/PIN sin correo electrónico.

Ejemplo:

kai
PIN ****

7.3. CHILD_PROFILE

Información específica de cada hijo.

Tabla `child_profiles`

Campo	Tipo
id	UUID PK

Campo	Tipo
user_id	UUID FK UNIQUE
display_name	VARCHAR(100)
birth_date	DATE
avatar	VARCHAR(255)
color_theme	VARCHAR(20)
allowance_enabled	BOOLEAN
screen_time_enabled	BOOLEAN
active	BOOLEAN
created_at	TIMESTAMP
updated_at	TIMESTAMP

La edad se calcula a partir de `birth_date`.

Nunca se almacena como valor fijo.

8. SISTEMA DE PAGA

8.1. ALLOWANCE_RULE

Permite establecer la paga.

Tabla `allowance_rules`

Campo	Tipo
id	UUID PK
family_id	UUID FK
child_id	UUID FK nullable
min_age	INTEGER nullable
max_age	INTEGER nullable
monthly_amount	DECIMAL(10,2)
spending_percentage	DECIMAL(5,2)

Campo	Tipo
savings_percentage	DECIMAL(5,2)
effective_from	DATE
effective_to	DATE nullable
active	BOOLEAN
created_at	TIMESTAMP
updated_at	TIMESTAMP

Ejemplo:

8-9 años
10 €
70 % gastar
30 % ahorrar

Validaciones

```
monthly_amount >= 0
spending_percentage >= 0
savings_percentage >= 0
spending_percentage + savings_percentage = 100
```

8.2. Prioridad de reglas

El sistema debe permitir:

Regla general por edad

Ejemplo:

8-9 → 10 €
10-11 → 15 €
12-13 → 20 €

Regla personalizada

Ejemplo:

Martí → 22 €

La regla personalizada prevalece sobre la regla general.

8.3. MONTHLY_ALLOWANCE

Registra la paga generada cada mes.

Tabla `monthly_allowances`

Campo	Tipo
id	UUID PK
child_id	UUID FK
year	INTEGER
month	INTEGER
gross_amount	DECIMAL(10,2)
spending_amount	DECIMAL(10,2)
savings_amount	DECIMAL(10,2)
allowance_rule_id	UUID FK
status	ENUM
generated_at	TIMESTAMP
confirmed_at	TIMESTAMP
confirmed_by	UUID FK

Status

DRAFT
CONFIRMED
CANCELLED

Restricción:

UNIQUE(child_id, year, month)

Esto impide generar dos pagas para el mismo mes.

8.4. Generación de paga

Ejemplo:

Paga:

20 €

Configuración:

70 % gasto
30 % ahorro

Generación:

MoneyTransaction
+14 € SPENDING

MoneyTransaction
+6 € SAVINGS

9. MONEDERO DE DINERO

9.1. MONEY_TRANSACTION

Tabla `money_transactions`

Campo	Tipo
id	UUID PK
child_id	UUID FK
wallet_type	ENUM
transaction_type	ENUM
amount	DECIMAL(10,2)
description	VARCHAR(255)
source_type	ENUM
source_id	UUID nullable
created_by	UUID FK
created_at	TIMESTAMP
reversed_transaction_id	UUID nullable

WalletType

SPENDING
SAVINGS

TransactionType

CREDIT
DEBIT

SourceType

```
MONTHLY_ALLOWANCE  
TASK  
BONUS  
PENALTY  
PURCHASE  
SAVINGS_TRANSFER  
MANUAL_ADJUSTMENT  
SETTLEMENT  
REVERSAL
```

9.2. Saldo

Dinero disponible:

```
SUM(  
  CASE  
    WHEN transaction_type = 'CREDIT' THEN amount  
    ELSE -amount  
  END  
)  
WHERE wallet_type = 'SPENDING'
```

Ahorro:

igual con `wallet_type = SAVINGS`.

9.3. Correcciones

Un movimiento confirmado NO debe modificarse ni borrarse.

Debe crearse un movimiento inverso.

Ejemplo:

```
+3 € TASK
```

Si fue incorrecto:

-3 € REVERSAL

Así se conserva auditoría.

10. TRANSFERENCIAS ENTRE DISPONIBLE Y AHORRO

Opcionalmente el adulto puede permitir:

DINERS → ESTALVIS

Tabla no necesaria: se generan dos movimientos.

Ejemplo:

Transferir 5 €.

SPENDING -5 €
SAVINGS +5 €

Ambos comparten:

transfer_reference_id

11. OBJETIVOS DE AHORRO

Tabla `savings_goals`

Campo	Tipo
id	UUID
child_id	UUID
name	VARCHAR(150)
target_amount	DECIMAL(10,2)
image_url	VARCHAR(255) nullable

Campo	Tipo
status	ENUM
created_at	TIMESTAMP
completed_at	TIMESTAMP nullable

Status

ACTIVE
COMPLETED
CANCELLED

Ejemplo:

Nintendo Switch
300 €
126 € acumulados
42 %

12. SISTEMA DE TAREAS

Debe diferenciarse entre:

RESPONSIBILITY

Responsabilidad habitual.

Ejemplo:

- fer el llit;
- preparar la motxilla;
- recollir l'habitació.

EXTRA

Trabajo adicional.

Ejemplo:

- rentar el cotxe;
- netejar terrassa;

- ordenar traster.

12.1. TASK

Tabla `tasks`

Campo	Tipo
id	UUID PK
family_id	UUID FK
name	VARCHAR(150)
description	TEXT
task_type	ENUM
icon	VARCHAR(50)
active	BOOLEAN
requires_approval	BOOLEAN
repeatable	BOOLEAN
recurrence_type	ENUM
max_completions_per_period	INTEGER nullable
created_by	UUID FK
created_at	TIMESTAMP
updated_at	TIMESTAMP

TaskType

RESPONSIBILITY
EXTRA

12.2. Recompensas de tarea

Una tarea puede proporcionar simultáneamente:

dinero
ahorro
pantalla

Tabla task_rewards

Campo	Tipo
id	UUID
task_id	UUID FK
money_amount	DECIMAL(10,2)
savings_amount	DECIMAL(10,2)
screen_minutes	INTEGER
active	BOOLEAN

Ejemplo:

Rentar coche

money_amount = 3
savings_amount = 0
screen_minutes = 15

12.3. Asignación de tareas

Tabla task_assignments

Campo	Tipo
id	UUID
task_id	UUID
child_id	UUID
active	BOOLEAN
available_from	DATE nullable
available_until	DATE nullable

Una tarea puede asignarse a uno o varios hijos.

12.4. Recurrencia

RecurrenceType

NONE
DAILY
WEEKLY
MONTHLY
CUSTOM

Ejemplos:

Fer el llit → DAILY

Rentar cotxe → WEEKLY

Ordenar traster → NONE

13. COMPLETADO DE TAREAS

Tabla `task_completions`

Campo	Tipo
id	UUID
task_id	UUID
child_id	UUID
completed_at	TIMESTAMP
status	ENUM
submitted_comment	VARCHAR(255)
reviewed_by	UUID nullable
reviewed_at	TIMESTAMP nullable
review_comment	VARCHAR(255) nullable

Campo	Tipo
reward_money	DECIMAL(10,2)
reward_savings	DECIMAL(10,2)
reward_screen_minutes	INTEGER
created_at	TIMESTAMP

CompletionStatus

PENDING
APPROVED
REJECTED
CANCELLED

13.1. Snapshot de recompensa

Muy importante:

Cuando el hijo marca una tarea hecha:

reward_money
reward_savings
reward_screen_minutes

se copian desde la configuración vigente.

Ejemplo:

Hoy:

Rentar cotxe = 3 €

Mañana el adulto cambia:

Rentar cotxe = 4 €

Las tareas antiguas continúan mostrando 3 €.

13.2. Aprobar tarea

Transacción backend:

```
BEGIN

TaskCompletion → APPROVED

crear MoneyTransaction si money > 0

crear MoneyTransaction SAVINGS si savings > 0

crear ScreenTimeTransaction si minutos > 0

COMMIT
```

Todo debe ejecutarse de forma atómica.

14. SISTEMA DE PANTALLAS

La pantalla funcionará como un monedero de minutos.

14.1. SCREEN_TIME_RULE

Tabla `screen_time_rules`

Campo	Tipo
id	UUID
child_id	UUID
weekday	INTEGER nullable
base_minutes	INTEGER
maximum_minutes	INTEGER nullable
rollover_enabled	BOOLEAN

Campo	Tipo
rollover_max_minutes	INTEGER nullable
valid_from	DATE
valid_until	DATE nullable
active	BOOLEAN

Permite configurar:

Dilluns-divendres → 45 min

Dissabte → 90 min

Diumenge → 90 min

14.2. SCREEN_TIME_TRANSACTION

Tabla screen_time_transactions

Campo	Tipo
id	UUID
child_id	UUID
transaction_type	ENUM
minutes	INTEGER
description	VARCHAR(255)
source_type	ENUM
source_id	UUID nullable
created_by	UUID
created_at	TIMESTAMP
reversed_transaction_id	UUID nullable

TransactionType

CREDIT
DEBIT

SourceType

DAILY_BASE
TASK
BONUS
PENALTY
USAGE
MANUAL_ADJUSTMENT
REVERSAL

15. GENERACIÓN DEL TIEMPO DIARIO

Cada día se genera:

DAILY_BASE +45 min

Debe existir una restricción idempotente:

child_id + date + DAILY_BASE

para impedir generar dos veces el tiempo base.

16. CONSUMO DE PANTALLA

El adulto puede registrar:

Ha gastat 30 minuts

Movimiento:

USAGE
-30 min

Opcionalmente se puede crear:

Iniciar pantalla

► Començar

y:

■ Acabar

El sistema calcularía automáticamente los minutos.

Esto sería una funcionalidad de segunda fase.

17. BONUS Y PENALIZACIONES

Sistema independiente de tareas.

Ejemplo:

Bona actitud aquesta setmana
+15 min

o:

No ha complert una responsabilitat
-10 min

Tabla **adjustments**

Campo	Tipo
id	UUID

Campo	Tipo
child_id	UUID
adjustment_type	ENUM
money_amount	DECIMAL
savings_amount	DECIMAL
screen_minutes	INTEGER
reason	VARCHAR(255)
created_by	UUID
created_at	TIMESTAMP

AdjustmentType

BONUS
PENALTY
MANUAL

18. RESPONSABILIDADES Y PANTALLA

Puede añadirse una regla opcional:

Pantalla bloqueada mientras existan responsabilidades obligatorias pendientes.

Ejemplo:

✓ Deures
✓ Motxilla
✓ Habitació
x Dutxa

Resultado:

Pantalla disponible: 55 min
Acceso: BLOQUEJAT

Cuando termina:

Pantalla: DISPONIBLE

Importante:

saldo de minutos y **permiso para usarlos** son conceptos diferentes.

19. HISTORIAL DE DINERO

Pantalla:

Moviments

Filtros:

Tots
Paga
Tasques
Bonificacions
Compres
Estalvis

Ejemplo:

18 ag.
Rentar el cotxe
+3 €

16 ag.
Paga mensual
+14 €

16 ag.
Estalvi mensual
+6 €

20. REGISTRO DE GASTOS

El niño puede comunicar un gasto y el adulto registrarlo.

Endpoint

```
POST /children/{id}/expenses
```

Datos:

```
{
  "amount": 7.50,
  "description": "Cromos"
}
```

Genera:

```
SPENDING
DEBIT
7,50 €
```

21. CIERRE MENSUAL

Elemento fundamental.

Tabla `monthly_settlements`

Campo	Tipo
id	UUID
child_id	UUID
year	INTEGER
month	INTEGER
base_allowance	DECIMAL
extra_earnings	DECIMAL

Campo	Tipo
bonuses	DECIMAL
penalties	DECIMAL
savings	DECIMAL
payable_amount	DECIMAL
status	ENUM
closed_at	TIMESTAMP
paid_at	TIMESTAMP nullable
closed_by	UUID

Status

OPEN
CLOSED
PAID
REOPENED

21.1. Resumen

Ejemplo:

Resum d'agost — Martí

Paga fixa	20,00 €
Tasques extra	+11,00 €
Bonificacions	+2,00 €
Generat	33,00 €
A estalvis	-6,00 €
A pagar	27,00 €

Botón:

21.2. Qué significa "pagado"

La aplicación no mueve dinero bancario.

Registra que el adulto ha efectuado físicamente/Bizum/transferencia/efectivo correspondiente.

En una primera versión NO se recomienda integrar banca real.

22. DASHBOARD DEL ADULTO

Ruta:

/dashboard

Contenido:

Resumen familiar

Cada hijo:

Martí
18,00 € disponible
126,00 € estalvis
45 min pantalla
2 pendants

Pendientes de aprobación

Rentar cotxe
Martí

+3 €
+15 min

[Rebutjar] [Aprovar]

Alertas

2 tasques pendents
1 tancament mensual pendent

23. DASHBOARD INFANTIL

Ruta:

/me

Diseño simplificado.

Hola, Kai 🖐️

ELS MEUS DINERS
12,50 €

ELS MEUS ESTALVIS
58,00 €

PANTALLA
35 min

TASQUES
3 disponibles

24. NAVEGACIÓN ADULTO

Inici
Infants
Tasques
Aprovacions

- Pagues
- Estalvis
- Pantalles
- Moviments
- Resums mensuals
- Configuració

25. NAVEGACIÓN INFANTIL

- Inici
- Tasques
- Diners
- Estalvis
- Pantalla
- Historial

26. CRUD — FAMILY

CREATE

Crear familia durante onboarding.

```
POST /api/families
```

READ

```
GET /api/families/current
```

UPDATE

```
PATCH /api/families/current
```

DELETE

No eliminación física inmediata.

```
DELETE /api/families/current
```

Debe iniciar proceso de borrado/anulación.

27. CRUD — CHILDREN

CREATE

```
POST /api/children
```

READ

```
GET /api/children  
GET /api/children/{id}
```

UPDATE

```
PATCH /api/children/{id}
```

DELETE

```
DELETE /api/children/{id}
```

Implementar como:

```
active = false
```

si existen movimientos históricos.

28. CRUD — ALLOWANCE RULES

```
POST /api/allowance-rules  
GET /api/allowance-rules  
GET /api/allowance-rules/{id}
```



```
PATCH /api/allowance-rules/{id}
DELETE /api/allowance-rules/{id}
```

29. CRUD — TASKS

```
POST /api/tasks
GET /api/tasks
GET /api/tasks/{id}
PATCH /api/tasks/{id}
DELETE /api/tasks/{id}
```

Filtros:

```
?type=EXTRA
?type=RESPONSIBILITY
?active=true
?childId=
```

30. CRUD — TASK ASSIGNMENTS

```
POST /api/tasks/{taskId}/assignments
GET /api/tasks/{taskId}/assignments
DELETE /api/tasks/{taskId}/assignments/{childId}
```

31. TASK COMPLETIONS

Hijo marca tarea:

```
POST /api/tasks/{taskId}/complete
```

Adulto aprueba:

```
POST /api/task-completions/{id}/approve
```

Adulto rechaza:

```
POST /api/task-completions/{id}/reject
```

Consultar:

```
GET /api/task-completions
```

Filtros:

```
status
childId
from
to
```

32. CRUD — SAVINGS GOALS

```
POST /api/children/{childId}/savings-goals
GET /api/children/{childId}/savings-goals
GET /api/savings-goals/{id}
PATCH /api/savings-goals/{id}
DELETE /api/savings-goals/{id}
```

33. MONEY API

Saldo

```
GET /api/children/{id}/wallet
```

Respuesta:

```
{
  "spendingBalance": 18.50,
  "savingsBalance": 126.00,
```

```
"total": 144.50
}
```

Movimientos

```
GET /api/children/{id}/money-transactions
```

Ajuste manual

```
POST /api/children/{id}/money-adjustments
```

Gasto

```
POST /api/children/{id}/expenses
```

Transferencia a ahorro

```
POST /api/children/{id}/savings-transfer
```

34. SCREEN TIME API

Estado actual

```
GET /api/children/{id}/screen-time/today
```

Respuesta:

```
{
  "baseMinutes": 45,
  "earnedMinutes": 25,
```

```
"penaltyMinutes": 10,  
"usedMinutes": 30,  
"availableMinutes": 30,  
"locked": false  
}
```

Añadir minutos

```
POST /api/children/{id}/screen-time/adjustments
```

Registrar consumo

```
POST /api/children/{id}/screen-time/usage
```

Movimientos

```
GET /api/children/{id}/screen-time/transactions
```

35. MONTHLY ALLOWANCE API

Generar paga:

```
POST /api/allowances/generate
```

Consultar:

```
GET /api/allowances
```

Confirmar:

```
POST /api/allowances/{id}/confirm
```

Cancelar:

```
POST /api/allowances/{id}/cancel
```

36. MONTHLY SETTLEMENT API

```
GET /api/settlements
GET /api/settlements/{id}

POST /api/settlements/generate

POST /api/settlements/{id}/close

POST /api/settlements/{id}/mark-paid

POST /api/settlements/{id}/reopen
```

37. AUTH API

```
POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout
GET /api/auth/me
```

38. LOGIN ADULTO

```
email
password
```

39. LOGIN INFANTIL

Se recomienda simplificar.

Ejemplo:

```
Familia
  ↓
Selecciona perfil

Martí
Pau
Kai

PIN: ****
```

Nunca mostrar un selector de niños de otras familias.

40. JWT

Access token corto.

Ejemplo:

```
15 minutos
```

Refresh token más largo.

El refresh token debe almacenarse preferiblemente en cookie:

```
HttpOnly
Secure
SameSite
```

No almacenar contraseña ni refresh token en `localStorage`.

41. CONTROL DE AUTORIZACIÓN

Nunca confiar en:

```
childId
```

enviado por el frontend.

Backend debe verificar siempre:

```
requestedChild.familyId == authenticatedUser.familyId
```

Para CHILD:

```
requestedChild.userId == authenticatedUser.id
```

42. SEGURIDAD DE ENDPOINTS

Ejemplo:

```
@PreAuthorize("hasRole('PARENT')")
```

para:

```
crear tarea  
editar tarea  
aprobar tarea  
ajustar dinero  
ajustar pantalla  
cerrar mes
```

43. VALIDACIONES

Dinero

Nunca utilizar:

```
double  
float
```

Usar:

BigDecimal

Minutos

Usar:

Integer

Importes

Máximo configurable.

Ejemplo:

$0 \leq \text{taskReward} \leq 999.99$

44. REGLAS CRÍTICAS DE NEGOCIO

RN-01

Una tarea aprobada solo puede recompensarse una vez.

RN-02

Una tarea rechazada no genera movimientos.

RN-03

Una tarea aprobada no puede aprobarse nuevamente.

RN-04

Una paga mensual solo puede generarse una vez por hijo/mes.

RN-05

Las recompensas históricas no cambian al modificar el precio actual de una tarea.

RN-06

No se eliminan movimientos financieros confirmados.

RN-07

Una corrección se realiza mediante reversión.

RN-08

Un hijo nunca puede acceder a información de otro.

RN-09

Un adulto solo puede acceder a miembros de su propia familia.

RN-10

El ahorro nunca se considera automáticamente dinero a pagar físicamente.

RN-11

La pantalla puede tener saldo positivo y estar bloqueada por reglas.

RN-12

Todas las operaciones que generen varios movimientos deben ser transaccionales.

45. CASO DE USO COMPLETO

Martí dispone de:

Paga fija: 20 €
Ahorro: 30 %

El día 1 se genera:

+14 € SPENDING
+6 € SAVINGS

Después realiza:

Rentar cotxe
+3 €
+15 min

Martí pulsa:

FETA

Estado:

PENDING

Adulto:

APROVAR

Backend genera:

MoneyTransaction +3 €
ScreenTimeTransaction +15 min

Saldo:

Disponible: 17 €
Ahorro: 6 €
Pantalla adicional: 15 min

46. CRUD DE CONFIGURACIÓN

Family settings

GET /api/settings
PATCH /api/settings

Configuraciones:

currency
timezone
language
allowNegativeBalance
allowSavingsTransfer
taskApprovalRequired
childCanSubmitExpenses
screenRolloverEnabled
monthlyClosingDay

47. CONFIGURACIÓN DE PANTALLA

Debe poder modificarse individualmente.

Pantalla:

Configuració de pantalla — Kai

Dilluns	45 min
Dimarts	45 min
Dimecres	45 min
Dijous	45 min
Divendres	60 min
Dissabte	90 min
Diumenge	90 min

Opciones:

Acumular minuts sobrants
Límit acumulació
Límit màxim diari
Bloquejar amb responsabilitats pendents

48. PANEL DE GESTIÓN DE TAREAS

Vista:

Tasques

Tabla:

Tasca	Tipus	Recompensa	Assignació	Estat
Rentar cotxe	Extra	3 €	Tots	Activa
Habitació	Responsabilitat	—	Kai	Activa
Jardí	Extra	2 € +15 min	Martí/Pau	Activa

Acciones:

Crear
Editar
Duplicar
Desactivar
Eliminar

Preferir desactivar frente a eliminar cuando tenga historial.

49. PANTALLA CREAR TAREA

Campos:

Nom
Descripció
Tipus
Icona
Infants
Repetició
Recompensa diners
Recompensa estalvis
Recompensa pantalla
Requereix aprovació
Data inici
Data final

50. APROBACIONES

Ruta:

/aprovacions

Tarjetas:

Martí

Rentar el cotxe

Avui 17:32

+3 €

+15 min

[Rebutjar]

[Aprovar]

Permitir aprobación múltiple:

Aprovar seleccionades

51. ESTADÍSTICAS

Primera versión:

Dinero

Ganado este mes

Ahorrado

Gastado

Tareas

Pantalla

Minutos base
Ganados
Penalizados
Consumidos

Tareas

Completadas
Pendientes
Rechazadas

52. GRÁFICOS

Para el niño:

Evolució dels meus estalvis

Gen ■■■
Feb ■■■■
Mar ■■■■■
...

Para adulto:

Diners generats per mes

Temps de pantalla

Tasques completades

No convertir el producto en un sistema competitivo entre hermanos.

No crear rankings de hijos.

53. NOTIFICACIONES INTERNAS

Tabla:

notifications

Campo	Tipo
id	UUID
user_id	UUID
type	VARCHAR
title	VARCHAR
message	TEXT
read	BOOLEAN
created_at	TIMESTAMP

Ejemplos:

Martí ha completat una tasca.

La paga d'agost està preparada.

Tens una tasca aprovada.

54. AUDITORÍA

Tabla `audit_logs`

Campo	Tipo
id	UUID
family_id	UUID
user_id	UUID
action	VARCHAR
entity_type	VARCHAR
entity_id	UUID
old_value	JSONB
new_value	JSONB

Campo	Tipo
created_at	TIMESTAMP
ip_address	VARCHAR nullable

Registrar operaciones sensibles:

```
modificación paga
cambio recompensa
aprobación tarea
rechazo
ajuste manual
penalización
cierre mensual
reapertura
```

55. ÍNDICES DE BASE DE DATOS

Crear índices como mínimo:

```
money_transactions(child_id, created_at);

screen_time_transactions(child_id, created_at);

task_completions(child_id, status);

task_completions(task_id);

monthly_allowances(child_id, year, month);

monthly_settlements(child_id, year, month);

task_assignments(child_id);

notifications(user_id, read);
```

56. BORRADO LÓGICO

Entidades con histórico:


```
Child
Task
AllowanceRule
User
```

deben usar:

```
active = false
```

No eliminar registros relacionados con movimientos.

57. MIGRACIONES

Utilizar:

Flyway

Nunca:

```
ddl-auto: update
```

en producción.

Producción:

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate
```

Los cambios de esquema deben hacerse con migraciones:

```
V1__initial_schema.sql
V2__add_savings_goals.sql
V3__add_screen_rules.sql
```

58. PAQUETES BACKEND

```
com.familiaplus
|
├─ auth
├─ family
├─ user
├─ child
├─ allowance
├─ task
├─ money
├─ savings
├─ screentime
├─ settlement
├─ notification
├─ audit
├─ security
├─ common
└─ config
```

Cada módulo:

```
controller
service
repository
entity
dto
mapper
exception
```

59. DTO

Nunca devolver directamente entidades JPA.

Ejemplo:

```
ChildResponseDTO
CreateChildRequestDTO
UpdateChildRequestDTO
```

60. MANEJO GLOBAL DE ERRORES

Implementar:

```
@RestControllerAdvice
```

Respuesta estándar:

```
{
  "timestamp": "2026-08-18T20:15:00Z",
  "status": 400,
  "code": "TASK_ALREADY_APPROVED",
  "message": "Aquesta tasca ja està aprovada",
  "path": "/api/task-completions/..."
}
```

61. CÓDIGOS DE ERROR DE DOMINIO

Ejemplos:

```
CHILD_NOT_FOUND
TASK_NOT_FOUND
TASK_NOT_ASSIGNED
TASK_ALREADY_COMPLETED
TASK_ALREADY_APPROVED
TASK_ALREADY_REJECTED
INVALID_ALLOWANCE_RULE
ALLOWANCE_ALREADY_GENERATED
INSUFFICIENT_BALANCE
INVALID_SCREEN_TIME
SETTLEMENT_ALREADY_CLOSED
ACCESS_DENIED
```

62. FRONTEND — ESTRUCTURA

```
src/  
|  
├─ assets/  
├─ components/  
├─ layouts/  
|   ├─ ParentLayout.vue  
|   └─ ChildLayout.vue  
├─ views/  
|   ├─ parent/  
|   └─ child/  
├─ router/  
├─ stores/  
├─ services/  
├─ composables/  
├─ types/  
├─ utils/  
└─ i18n/
```

63. STORES PINIA

```
authStore  
familyStore  
childrenStore  
taskStore  
moneyStore  
savingsStore  
screenTimeStore  
approvalStore  
settlementStore  
notificationStore
```

Pinia está diseñado como store para estado global y lógica compartida en aplicaciones Vue.

64. VISTAS ADULTO

```
ParentDashboardView
ChildrenView
ChildDetailView
TasksView
TaskFormView
ApprovalsView
AllowanceView
SavingsView
ScreenTimeView
TransactionsView
SettlementsView
SettingsView
```

65. VISTAS CHILD

```
ChildDashboardView
MyTasksView
MyMoneyView
MySavingsView
MyScreenTimeView
MyHistoryView
MyGoalsView
```

66. DISEÑO RESPONSIVE

Prioridad:

```
mobile first
```

Breakpoints aproximados:

```
Mobile
< 768

Tablet
```

768-1024

Desktop
>1024

67. PWA

Posteriormente puede instalarse como:

app en pantalla de inicio

sin necesidad inicial de publicar en App Store o Google Play.

Debe poder evolucionar a PWA.

68. UX INFANTIL

Principios:

poco texto
números grandes
iconos
acciones claras
feedback inmediato
sin tablas complejas

Ejemplo:

 126 €

Els meus estalvis

69. UX ADULTO

Prioridad:

```
control
visión global
pocos clics
historial
acciones rápidas
```

70. IDIOMA

Todo texto visible:

Català

Ejemplos:

```
Inici
Tasques
Diners
Estalvis
Pantalla
Moviments
Aprovacions
Configuració
Aprovar
Rebutjar
Marcar com a feta
```

El código puede estar en inglés.

Ejemplo correcto:

```
SavingsGoal
TaskCompletion
MoneyTransaction
```

No:

```
ObjectiuEstalvi
TascaCompletada
```

71. INTERNACIONALIZACIÓN

Aunque inicialmente solo catalán:

```
vue-i18n
```

Strings:

```
{
  "tasks": {
    "title": "Tasques",
    "completed": "Feta",
    "approve": "Aprovar"
  }
}
```

Así posteriormente se puede añadir castellano.

72. PRIVACIDAD

La aplicación manejará datos de menores.

Debe aplicarse desde el diseño:

```
mínima recogida de datos
no publicidad
no tracking innecesario
no venta de datos
perfiles privados
acceso familiar aislado
```

Los datos personales de menores reciben protección específica bajo el RGPD y la información dirigida a menores debe presentarse de forma clara y adaptada a su edad.

73. DATOS QUE NO NECESITAMOS

No recopilar:

dirección
colegio
teléfono del menor
ubicación
fotografías obligatorias
apellidos completos

Nombre visible puede ser:

Kai
Pau
Martí

74. CONTRASEÑAS

Backend:

BCrypt o Argon2

Nunca almacenar contraseñas en texto plano.

75. CORS

En producción únicamente permitir:

<https://app.familiaplus.es>

No:

*

76. HTTPS

Obligatorio en producción.

77. RATE LIMITING

Aplicar especialmente en:

```
/login  
/refresh
```

para reducir ataques de fuerza bruta.

78. LOGGING

Registrar:

```
errores  
login  
fallos de autenticación  
operaciones críticas
```

Nunca registrar:

```
password  
JWT  
PIN
```

79. BACKUPS

Producción:

```
backup automático diario PostgreSQL
```

Retención sugerida:

```
7 diarios
4 semanales
3 mensuales
```

80. HEALTH CHECK

Spring Boot Actuator:

```
/actuator/health
```

No exponer endpoints internos innecesarios públicamente.

81. TESTING BACKEND

Unit tests

Servicios:

```
AllowanceService
TaskService
MoneyService
ScreenTimeService
SettlementService
```

Integration tests

Usar:

```
Testcontainers PostgreSQL
```

82. TESTS CRÍTICOS

Debe comprobarse:

Una tarea aprobada genera recompensa.

Una segunda aprobación no genera otra recompensa.

Una tarea rechazada no genera saldo.

Una regla de paga genera porcentajes correctos.

No puede generarse paga duplicada.

Un niño no accede a otro niño.

Un adulto no accede a otra familia.

Una reversión corrige el saldo.

Un cierre mensual calcula correctamente.

Una operación parcialmente fallida hace rollback.

83. TESTING FRONTEND

Recomendación:

Vitest
Vue Test Utils

End-to-end:

Playwright

84. FLUJOS E2E IMPRESCINDIBLES

E2E-01

Adulto login
→ crear hijo
→ configurar paga

- generar paga
- comprobar saldo

E2E-02

Adulto crea tarea

- asigna niño
- niño login
- completa tarea
- adulto aprueba
- saldo aumenta

E2E-03

Tarea + pantalla

- aprobación
- minutos aumentan

E2E-04

Cerrar mes

- ver importe
- marcar pagado

85. DOCUMENTACIÓN API

Añadir:

springdoc-openapi

Disponible en desarrollo:

/swagger-ui

86. DOCKER

Servicios:

```
frontend
backend
database
```

Docker Compose permite definir y ejecutar aplicaciones multi-contenedor y contempla despliegues en un único host.

87. DOCKER COMPOSE DEV

```
PostgreSQL
Backend
Frontend
```

Desarrollo con hot reload.

88. PRODUCCIÓN

Arquitectura inicial recomendada:

```
Internet
|
HTTPS
|
Reverse Proxy
|
├── frontend
└── /api
    |
    backend
    |
    PostgreSQL
```

89. VARIABLES DE ENTORNO

Backend:

```
DB_URL
DB_USERNAME
DB_PASSWORD

JWT_SECRET
JWT_ACCESS_EXPIRATION
JWT_REFRESH_EXPIRATION

FRONTEND_URL

SPRING_PROFILES_ACTIVE
```

Frontend:

```
VITE_API_URL
```

Nunca subir `.env` real a Git.

90. PERFILES SPRING

```
application.yml
application-dev.yml
application-test.yml
application-prod.yml
```

Producción:

```
ddl-auto: validate
show-sql: false
```

91. CI/CD

GitHub Actions.

Pipeline Pull Request:

```
checkout
backend tests
frontend tests
frontend build
backend build
```

Main:

```
tests
build
Docker images
deploy
health check
```

92. RAMAS

Para un proyecto individual:

```
main
feature/*
fix/*
```

No es necesario Git Flow completo.

93. VERSIONADO

Releases:

```
v0.1.0
v0.2.0
v1.0.0
```

94. MVP — FASE 1

Debe contener:

Autenticación

Familia

Hijos

Paga

Separación gasto/ahorro

Tareas

Recompensas

Aprobación

Dinero

Pantalla

Historial

Dashboard

Cierre mensual

Con esto la aplicación ya es funcional.

95. FASE 2

Añadir:

Objetivos de ahorro

Tareas recurrentes

Responsabilidades

Bloqueo pantalla

Gráficas

Notificaciones

Transferencias ahorro

PWA

96. FASE 3

Añadir:

Rutinas

Calendario

Recompensas especiales

Insignias no competitivas

Objetivos familiares

Exportación PDF

Exportación CSV

97. FUNCIONES QUE NO DESARROLLARÍA EN EL MVP

No añadir inicialmente:

IA

conexión bancaria

Bizum

notificaciones push

control real del dispositivo

App nativa

gamificación compleja

chat

Primero debe funcionar perfectamente el núcleo.

98. POSIBLE CONTROL REAL DE PANTALLAS

La primera versión gestiona:

minutos concedidos
minutos consumidos

pero no bloquea físicamente:

iPad
Android
PlayStation
Nintendo
PC

Integrar el bloqueo efectivo requeriría APIs específicas de cada plataforma y debe considerarse otro proyecto.

99. CRITERIOS DE ACEPTACIÓN MVP

El producto puede considerarse MVP terminado cuando:

- ✓ adulto puede crear familia;
- ✓ puede crear hijos;
- ✓ cada hijo tiene acceso independiente;

- ✓ puede configurar paga;
- ✓ puede dividirla entre gasto y ahorro;
- ✓ puede generar paga mensual;
- ✓ el niño ve sus saldos;
- ✓ adulto crea tareas;
- ✓ puede asignarlas;
- ✓ puede establecer recompensa económica;
- ✓ puede establecer recompensa pantalla;
- ✓ niño puede marcar tarea;
- ✓ adulto puede aprobar/rechazar;
- ✓ aprobación actualiza saldos;
- ✓ hay histórico;
- ✓ se configura pantalla diaria;
- ✓ se añaden bonus;
- ✓ se añaden penalizaciones;
- ✓ se registra consumo;
- ✓ existe resumen mensual;
- ✓ puede marcarse como pagado;
- ✓ existe autorización por familia;
- ✓ existen tests;
- ✓ existen migraciones;
- ✓ aplicación funciona mediante HTTPS en producción;
- ✓ existen backups.

100. ROADMAP DE DESARROLLO

Sprint 0 — Preparación

Repositorio

Spring Boot

Vue

PostgreSQL

Docker

CI

Configuraciones

Sprint 1 — Auth y familia

User

Family

Roles

JWT

Login

Security

Sprint 2 — Hijos

ChildProfile

CRUD

Permisos

Dashboard base

Sprint 3 — Money Ledger

MoneyTransaction

saldo

historial

ajustes

gastos

Sprint 4 — Paga

AllowanceRule

MonthlyAllowance

generación

reparto ahorro/gasto

Sprint 5 — Tareas

Task

TaskReward

TaskAssignment

TaskCompletion

Sprint 6 — Aprobaciones

Pending

Approve

Reject

generación recompensa

Sprint 7 — Pantalla

ScreenTimeRule

ScreenTimeTransaction

saldo

bonus

penalización

consumo

Sprint 8 — Ahorro

Savings

Goals

Transfers

gráficos

Sprint 9 — Cierre mensual

Settlement

Close

Paid

Historical

Sprint 10 — UX

dashboard adulto

dashboard infantil

responsive

català

accesibilidad

Sprint 11 — QA

unit tests

integration

E2E

security

validaciones

Sprint 12 — Producción

Docker production

dominio

HTTPS

PostgreSQL producción

backup

monitorización

deploy

101. DEFINICIÓN DE DONE

Una funcionalidad solo está terminada cuando:

backend implementado

frontend implementado

validaciones

autorización

manejo de errores

test unitario

test integración si procede

responsive

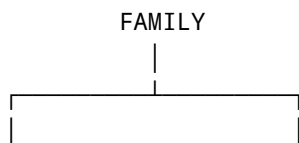
texto catalán

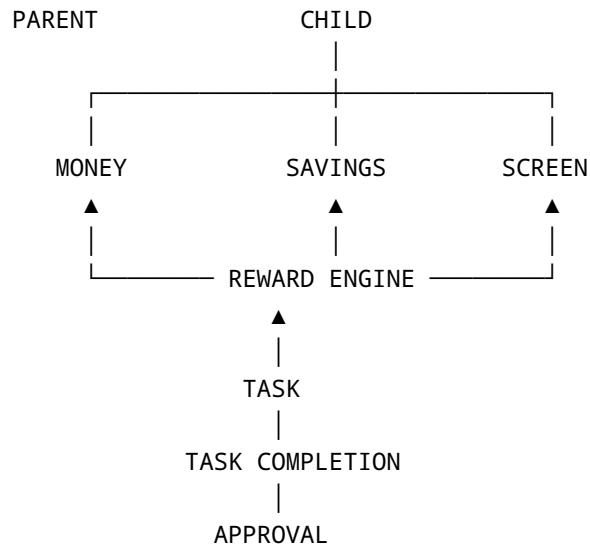
documentación actualizada

102. ERD SIMPLIFICADO

```
erDiagram
    FAMILY ||--o{ USER : contains
    USER ||--o| CHILD_PROFILE : has
    FAMILY ||--o{ ALLOWANCE_RULE : defines
    CHILD_PROFILE ||--o{ MONTHLY_ALLOWANCE : receives
    CHILD_PROFILE ||--o{ MONEY_TRANSACTION : owns
    CHILD_PROFILE ||--o{ SCREEN_TIME_TRANSACTION : owns
    CHILD_PROFILE ||--o{ SAVINGS_GOAL : has
    FAMILY ||--o{ TASK : defines
    TASK ||--o| TASK_REWARD : has
    TASK ||--o{ TASK_ASSIGNMENT : assigned
    CHILD_PROFILE ||--o{ TASK_ASSIGNMENT : receives
    TASK ||--o{ TASK_COMPLETION : creates
    CHILD_PROFILE ||--o{ TASK_COMPLETION : performs
    CHILD_PROFILE ||--o{ SCREEN_TIME_RULE : has
    CHILD_PROFILE ||--o{ MONTHLY_SETTLEMENT : has
    FAMILY ||--o{ AUDIT_LOG : generates
```

103. RELACIÓN ENTRE LOS MÓDULOS





104. REWARD ENGINE

Para evitar lógica duplicada debe existir un servicio:

RewardService

Responsabilidad:

```
receive reward definition
validate
generate money movements
generate savings movements
generate screen movements
persist atomically
```

Utilizado por:

TaskService
AdjustmentService
AllowanceService

105. SERVICIOS BACKEND PRINCIPALES

AuthService
FamilyService
ChildService
AllowanceRuleService
AllowanceService
TaskService
TaskCompletionService
RewardService
MoneyService
SavingsService
ScreenTimeService
SettlementService
NotificationService
AuditService

106. TRANSACCIONES

Usar:

```
@Transactional
```

especialmente en:

```
approveTask()  
generateAllowance()  
transferToSavings()  
closeMonthlySettlement()  
reverseTransaction()
```

107. CONCURRENCIA

Debe impedirse que dos peticiones simultáneas aprueben la misma tarea.

Opciones:

```
optimistic locking
```

añadiendo:

```
@Version
```

a:

```
TaskCompletion  
MonthlySettlement
```

108. IDEMPOTENCIA

Operaciones críticas:

generar paga

generar minutos diarios

aprobar tarea

cerrar mes

deben soportar reintentos sin duplicar movimientos.

109. DECIMALES Y REDONDEO

Ejemplo:

Paga = 17 €
Ahorro = 30 %

Resultado matemático:

5,10 €

Regla:

escala 2
HALF_UP

La suma:

spendingAmount + savingsAmount

debe ser exactamente igual a:

grossAmount

110. HORARIOS

Guardar timestamps de sistema en:

UTC

Mostrar al usuario en:

Europe/Madrid

111. FECHAS MENSUALES

Los cierres y pagas se basan en:

zona horaria de la familia

no en la del servidor.

112. ACCESIBILIDAD

Como mínimo:

contraste adecuado
navegación teclado
labels
focus visible
texto no dependiente únicamente del color
icono + texto

113. ESTILO VISUAL

La herramienta debe transmitir:

familia
claridad
autonomía
progreso

No:

banco corporativo
sistema disciplinario
panel empresarial

114. COLORES FUNCIONALES

Conceptualmente:

Diners → verde

Estalvis → azul

Pantalla → violeta

Tasques → naranja

Aprovació → verde

Pendent → amarillo

Rebutjat → rojo

Manteniendo siempre contraste accesible.

115. HOME INFANTIL IDEAL

Hola, Martí 🖐️

💰 18,50 €

Diners

 126,00 €
Estalvis

 45 min
Pantalla

Tasques disponibles

 Rentar el cotxe
+3 € · +15 min

[Marcar com a feta]

116. HOME ADULTO IDEAL

Bon vespre 🖐️

FAMÍLIA

Martí

18 € · 126 € · 45 min

Pau

11 € · 87 € · 30 min

Kai

7 € · 54 € · 40 min

PENDENTS D'APROVACIÓ

Martí

Rentar el cotxe

+3 € +15 min

[Aprovar]

117. FUTURA EXPORTACIÓN

Permitir:

CSV
PDF

por:

hijo
mes
año

118. MÉTRICAS TÉCNICAS

Monitorizar:

uptime

HTTP 5xx

response time

database connections

disk usage

backup status

No incorporar analytics de comportamiento infantil innecesarios.

119. DESPLIEGUE INICIAL RECOMENDADO

Para uso familiar real puede utilizarse:

```
1 VPS pequeño
Docker Compose
Reverse proxy
Backend
Frontend
PostgreSQL
Backup externo
```

Docker documenta Compose también para despliegues productivos sobre un único servidor.

Más adelante puede separarse:

```
frontend hosting

backend hosting

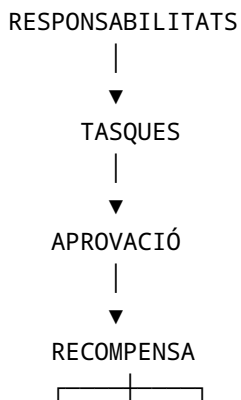
managed PostgreSQL
```

sin cambiar la arquitectura de la aplicación.

120. RESULTADO FINAL ESPERADO

Família+ no será únicamente una aplicación de paga.

El producto final será un sistema familiar conectado:





El núcleo técnico será:

ledger + reglas + aprobaciones + auditoría.

Esto permitirá modificar precios, pagas, porcentajes, minutos, tareas y condiciones sin perder nunca el histórico.

121. OBJETIVO DE LA VERSIÓN 1.0

La versión 1.0 debe permitir que una familia pueda utilizar la herramienta durante todo un mes sin necesidad de Excel, papel u otra aplicación para saber:

qué paga corresponde a cada hijo, cuánto puede gastar, cuánto tiene ahorrado, qué trabajos ha realizado, qué recompensas ha obtenido, cuánto tiempo de pantalla tiene disponible y cuál es el importe que corresponde cerrar o pagar al finalizar el mes.

Ese debe ser el criterio principal para considerar el producto preparado para producción.